



Auditoría — Wallet Control

21 de julio de 2026 · Auditoría de código y build (AUDITA) · Comparada contra la auditoría del 15 de julio de 2026

Resumen ejecutivo: Wallet Control está funcionalmente completa y con buena disciplina de código: cero errores de TypeScript, cero secrets expuestos, sin `console.log` sueltos, y las políticas de seguridad (RLS) de Supabase están bien configuradas. El proyecto ya está vinculado a EAS y listo para generar un build instalable en cualquier momento. Lo que falta para considerarla realmente lista para publicar es, sobre todo, un tema legal (la política de tratamiento de datos formal) y dos confirmaciones de "eliminar" que aún rompen el tema oscuro — mismo patrón ya corregido en el resto de la app, solo falta replicarlo en dos lugares puntuales.

● ~88% lista para publicar



Bugs / errores reales

Dependencia faltante rompía el entorno de desarrollo en esta máquina

Alta

Qué es: `@react-native-community/datetimepicker` estaba declarada en `package.json` y en el lockfile (agregada por tu hermano en la sesión del 2026-07-12 para el selector de fecha límite de metas), pero nunca se había instalado en esta máquina — el `git pull` trae el código, no `node_modules`.

Por qué importa: con la dependencia faltante, `npx tsc --noEmit` y `npx expo-doctor` fallaban por completo, y `npx expo start` también habría fallado al intentar levantar el servidor — bloqueaba cualquier desarrollo o build en este equipo.

Acción: ya se corrigió durante esta auditoría corriendo `npm install` (no se tocó ningún archivo de código). Para el futuro: correr `npm install` después de cada `git pull` cuando el historial trae cambios de dependencias.

"Eliminar depósito" en el historial de una meta todavía usa el diálogo nativo

Media

Qué es: en `app/(tabs)/resumen.tsx` (línea ~1677), al eliminar un depósito del historial de una meta de ahorro, todavía se dispara `Alert.alert` nativo de Android (fondo blanco) en vez del modal temático oscuro.

Por qué importa: es exactamente el mismo problema visual que ya se corrigió para "eliminar meta", "eliminar tarjeta/cuenta", "eliminar categoría" y "eliminar gasto" en las sesiones del 19 y 20 de julio — este caso puntual quedó fuera de esa limpieza.

Acción: aplicar el mismo patrón de modal de confirmación centrado (ícono de basurero, botones Cancelar/Eliminar en rojo) ya usado en los otros 4 lugares.

"Cerrar sesión" en Perfil también usa el diálogo nativo Media

Qué es: en `app/(tabs)/perfil.tsx` (línea 84), el botón "Cerrar sesión" abre un `Alert.alert` nativo con dos botones (Cancelar / destructivo), mismo patrón visual que rompe el tema oscuro.

Por qué importa: es una acción frecuente y visible; el contraste con el resto de la app (que ya usa modales propios) se nota.

Acción: reemplazar por el mismo modal de confirmación temático.

Para eliminar / actualizar

Nota desactualizada en `ajustes wallet-control.txt` Baja

Qué es: el archivo lista como "[PENDIENTE] por MAQUINA" hacer `eas login / eas init`, pero el proyecto ya está vinculado: `app.json` tiene `extra.eas.projectId` configurado, `eas.json` ya tiene los 3 perfiles de build, y esta máquina ya tiene sesión activa (`eas whoami` → `nicksero`).

Acción: mover ese ítem a RESUELTO (verificar si en la máquina del hermano también hay sesión activa antes de darlo por cerrado del todo ahí).

Archivos muy grandes, candidatos a dividir Baja

Qué es: `app/(tabs)/resumen.tsx` (1.947 líneas), `app/(tabs)/tarjetas.tsx` (1.096) y `app/onboarding.tsx` (1.012) concentran gran parte de la lógica de la app en un solo archivo cada uno.

Por qué importa: no es un bug — la app funciona bien así — pero a este tamaño cada nuevo ajuste toma más tiempo en ubicarse y probar.

Acción: si en algún momento el ritmo de cambios se siente más lento, vale la pena extraer secciones a componentes propios (como ya se hizo con `CategoryDetailModal.tsx`).

Mejoras recomendadas

Falta la Política de Tratamiento de Datos formal y publicada Alta

Qué es: hoy existe una sección de preguntas frecuentes amigable sobre privacidad dentro de la Guía de usuario (`app/ayuda.tsx`), pero no hay una política de tratamiento de datos formal publicada en una URL pública.

Por qué importa: es un requisito real para el formulario "Data safety" de Play Store, y en Colombia aplica la Ley 1581 de 2012 (Habeas Data) para cualquier app que maneje datos personales — ya está en el checklist de "proyecto terminado" del sistema.

Acción: redactar la política (puede basarse en el texto que ya existe en la FAQ) y publicarla en una página accesible por URL antes de subir la app a Play Store.

Accesibilidad parcial en componentes reutilizables Media

Qué es: hay 86 usos de `accessibilityLabel` / `Role` repartidos en 13 de 29 archivos `.tsx`, pero componentes interactivos reutilizados en toda la app — `BottomSheet.tsx`, `CardView.tsx`, `BudgetFormModal.tsx` — todavía no tienen ninguno.

Por qué importa: este era un hallazgo real de la auditoría del 2026-07-06; hubo avance real pero quedó incompleto en las piezas que se repiten en más pantallas.

Acción: agregar `accessibilityLabel` a los botones y campos de esos 3 componentes primero — cubren la mayor parte de la superficie táctil de la app.

20 vulnerabilidades reportadas por `npm audit` Baja

Qué es: 16 moderadas, 3 altas y 1 crítica — todas transitivas del toolchain de Expo SDK 54 (`tar`, `js-yaml`, `shell-quote`, `brace-expansion` dentro de `@expo/config-plugins`).

Por qué importa: son herramientas de build/desarrollo, no código que corre en el celular del usuario final — no son explotables en producción. Se resolverían saltando a Expo 57 (salto mayor, no trivial, no recomendado hoy mismo).

Acción: no bloquea el lanzamiento. Planear la migración a Expo 57 como tarea separada, no urgente.

Parche menor de Expo disponible Baja

Qué es: `expo` está en 54.0.35, la versión esperada por el SDK instalado es 54.0.36 (detectado por `expo-doctor`).

Acción: correr `npx expo install --check` antes del próximo build de producción para alinear el parche.

✓ Lo que ya funciona bien

✓ `npx tsc --noEmit` — 0 errores de TypeScript en todo el proyecto.

✓ RLS de Supabase en `profiles` correctamente configurada (select/insert/update solo del propio usuario), siguiendo el patrón recomendado.

✓ Sin secrets hardcodeados — `.env` está en `.gitignore` y no se encontró ninguna key expuesta en el código.

✓ Sin `console.log` sueltos en `app/`, `components/` ni `lib/` — buena disciplina de limpieza.

✓ El hallazgo de la auditoría del 2026-07-06 sobre el aviso de permisos de notificaciones denegados (`CardFormModal.tsx`) ya está resuelto en el código actual.

✓ Los reemplazos de `Alert.alert` por modales temáticos en "eliminar meta/tarjeta/categoría/gasto"

(v1.17, v1.18) están bien implementados y ya verificados con `tsc --noEmit` por tu hermano.

✅ Proyecto EAS ya vinculado (`projectId` en `app.json` , 3 perfiles de build en `eas.json`) y con sesión activa en esta máquina — listo para `eas build --profile preview` en cualquier momento.

✅ 16 de 18 checks de `expo-doctor` pasan; los 2 que fallan son un problema de red temporal (no bloqueante) y el desfase de parche menor ya mencionado.
